

Implementação e Análise Comparativa entre TCP e UDP Confiável em um Miniservidor Web com DNS Local sob Cenários de Perda e Latência Simuladas

Pedro Henrique Silva Rodrigues¹
(MATRÍCULA: 20249011446)

¹ Universidade Federal do Piauí (UFPI) – Campus Senador Helvídio
Nunes de Barros (CSHNB) Picos – PI – Brasil

`pedro.rodrigues.pr@ufpi.edu.br`

Resumo. *Este trabalho apresenta a implementação e análise comparativa entre os protocolos TCP e UDP Confiável (R-UDP) em um miniservidor web HTTP/1.1 com resolução DNS local. O R-UDP emprega um mecanismo Stop-and-Wait com numeração de sequência, reconhecimento (ACK), timeout, retransmissão em caso de perda e verificação de integridade na camada de aplicação. Ambos os modos de transporte foram submetidos a três cenários de rede simulados com a ferramenta tc netem em ambiente Docker para arquivos de diferentes tamanhos. Os resultados mostram que o TCP supera significativamente o R-UDP em todos os cenários, especialmente sob perda e latência elevadas.*

Palavras-chave: TCP; Reliable-UDP; Miniservidor; HTTP/1.1; DNS.

1. Introdução

As redes de computadores constituem a espinha dorsal dos sistemas modernos, viabilizando a comunicação entre dispositivos e sustentando aplicações que exigem entrega confiável de dados, baixa latência e utilização eficiente dos recursos de infraestrutura. Nesse contexto, a pilha de protocolos TCP/IP desempenha um papel fundamental, e serviços como a resolução de nomes e a transferência de arquivos são componentes críticos para o funcionamento da *World Wide Web*. A compreensão do comportamento desses serviços sob condições adversas de rede é essencial para o projeto de aplicações robustas e eficientes.

A comunicação entre cliente e servidor web envolve, em um cenário típico, duas etapas principais: a resolução do nome de domínio para um endereço IP, realizada pelo DNS (*Domain Name System*), e a transferência propriamente dita dos recursos HTTP, que depende de um protocolo de transporte confiável. Em aplicações reais, ambas as etapas estão sujeitas a condições de rede adversas, como latência variável e perda de pacotes, que podem comprometer severamente a experiência do usuário. A escolha do protocolo de transporte, como o TCP (*Transmission Control Protocol*) ou UDP (*User Datagram Protocol*) com confiabilidade implementada na camada de aplicação, influencia diretamente o desempenho e a robustez do sistema como um todo.

Este trabalho propõe a implementação de um miniservidor web HTTP/1.1 completo, integrado a um resolvedor DNS local, que oferece dois modos de transporte para

a transferência de arquivos: o TCP nativo e um protocolo UDP Confiável (R-UDP) baseado no mecanismo *Stop-and-Wait*. Ambos os modos foram submetidos a cenários de rede controlados simulados, variando atraso e taxa de perda de pacotes para três tamanhos de arquivos. A comparação fundamenta-se em métricas objetivas de vazão (*throughput*), tempo de transferência e taxa de erro, visando evidenciar as vantagens e limitações de cada abordagem no contexto de um sistema web funcional com resolução DNS local.

1.1. Objetivos

Este trabalho tem como objetivos:

1. **Implementar um resolvedor DNS local:** desenvolver um servidor e cliente DNS simplificados sobre UDP, capazes de traduzir nomes de domínio para endereços IP com mecanismo de *timeout* e retransmissão na camada de aplicação.
2. **Implementar um miniservidor web HTTP/1.1 com suporte a TCP e R-UDP:** construir um servidor web capaz de servir arquivos estáticos nos dois modos de transporte, utilizando enquadramento próprio e cabeçalho de autenticação X-Custom-Auth para validação cruzada com capturas de rede.
3. **Desenvolver um cliente web integrado a DNS e HTTP:** implementar um cliente que realize a resolução de nomes obrigatoriamente via DNS local antes de efetuar a requisição HTTP, encapsulando o fluxo completo de uma transferência web realista.
4. **Simular cenários adversos de rede:** reproduzir condições controladas de latência e perda de pacotes utilizando as ferramentas *tc* / *netem* em ambiente Docker, abrangendo três cenários: baixa latência sem perdas, latência média com perda moderada e alta latência com perda elevada.
5. **Comparar o desempenho entre TCP e R-UDP no contexto completo:** analisar e contrastar os resultados de vazão, duração e taxa de erro obtidos em todos os cenários e tamanhos de arquivo, identificando as vantagens e limitações de cada abordagem dentro de um sistema web funcional com resolução DNS local.

1.2. Escopo e Delimitações

O escopo deste trabalho está delimitado aos seguintes aspectos:

- **Sistema implementado:** foi desenvolvido um miniservidor web HTTP/1.1 completo, composto por um servidor e um cliente web, integrado a um resolvedor DNS local simplificado. O cliente realiza obrigatoriamente a resolução do nome de domínio via DNS antes de efetuar a requisição HTTP, reproduzindo o fluxo típico de uma transferência web realista.
- **Protocolos de transporte:** o servidor e o cliente web operam em dois modos de transporte distintos: (i) TCP nativo, utilizando a implementação da biblioteca-padrão do Python com `socket.create_connection`; e (ii) R-UDP, uma camada de confiabilidade sobre UDP implementada na aplicação, baseada no algoritmo *Stop-and-Wait* com numeração de sequência, ACK, temporizador fixo de 0,5 segundo, retransmissão de pacotes (até 1.000 tentativas), e verificação de integridade por CRC32.
- **Cabeçalho de autenticação:** todas as mensagens entre cliente e servidor incluem um cabeçalho X-Custom-Auth (SHA-256) contendo matrícula e nome do aluno, permitindo a identificação e validação dos fluxos nas capturas de rede realizadas com TCPDump e Wireshark/tshark.

- **Infraestrutura de experimentos:** os experimentos são executados em ambiente Docker Compose v2 com três *containers* interconectados por uma rede interna: servidor DNS (na porta 53/UDP), servidor web (na porta 8080) e cliente web. O `tc qdisc netem` é aplicado diretamente no *container* do servidor web para simular as condições de rede.
- **Delimitações:** o trabalho não abrange a implementação de mecanismos avançados de controle de congestionamento, janela deslizante ou timeout adaptativo no R-UDP, limitando-se a uma camada de confiabilidade *Stop-and-Wait* simples. O resolvidor DNS implementado é funcional, porém simplificado, não suporta registros além do tipo A, cache ou consultas recursivas completas. Os experimentos são conduzidos em ambiente local controlado (Docker), não refletindo necessariamente o comportamento em redes de longa distância ou na Internet real.

1.3. Questões a Serem Respondidas

Este trabalho busca responder às seguintes questões fundamentais:

1. Como a perda de pacotes simulada no canal afetou o tempo de resolução DNS (que usa UDP nativo sem retransmissão na camada de transporte) em comparação com o download da página via HTTP (R-UDP/TCP)? O cliente DNS precisou implementar algum *timeout* na aplicação?
2. Qual foi o impacto visual (na análise de gráficos) e métrico do *overhead* dos cabeçalhos HTTP adicionados nesta avaliação, quando comparado ao protocolo de aplicação puramente textual e customizado da Segunda Avaliação?
3. Ao inspecionar o Wireshark, o fluxo de pacotes seguiu estritamente a ordem esperada da arquitetura da Internet? Demonstre através de *prints* do fluxo de tempo do Wireshark a transição exata entre o encerramento da *query* DNS e o início do *handshake* TCP / primeira transmissão R-UDP.

2. Fundamentação Teórica

Esta seção apresenta os conceitos fundamentais necessários para a compreensão do trabalho, abordando o DNS e o protocolo HTTP/1.1, bem como os protocolos de transporte TCP e R-UDP utilizados na transferência de dados entre cliente e servidor. Esses fundamentos fornecem a base teórica para a análise comparativa realizada nos experimentos.

2.1. DNS (Domain Name System)

O DNS é um serviço de camada de aplicação responsável por traduzir nomes de domínio legíveis por humanos em endereços IP numéricos utilizados pelos roteadores e *hosts* para estabelecer comunicação em redes IP. O DNS adota uma arquitetura hierárquica e distribuída, organizada em zonas de autoridade. Quando um cliente deseja resolver um nome, ele envia uma consulta a um resolvidor DNS, que percorre a hierarquia de servidores até obter a resposta.

Na prática, no entanto, o protocolo DNS opera sobre UDP na porta 53, com uma camada simplificada de requisição-resposta: o cliente envia um pacote contendo um identificador único e o nome a ser resolvido, e o servidor responde com o endereço IP correspondente. Neste trabalho, foi implementado um resolvidor DNS simplificado utilizando um formato de pacote próprio, com campos para identificador da consulta, tamanho do nome, nome codificado em UTF-8 e, no caso da resposta, o endereço IPv4 de 4 bytes.

O servidor DNS mantém uma tabela de resolução local carregada a partir de um arquivo de zona (`hosts.txt`), permitindo associar nomes de domínio a endereços IP sem recorrer a servidores externos. O cliente DNS, por sua vez, implementa um mecanismo de *timeout* e retransmissão na camada de aplicação, enviando até 5 tentativas com intervalo de 2 segundos, uma vez que o UDP não oferece garantia de entrega.

2.2. HTTP/1.1 (Hypertext Transfer Protocol)

O HTTP é o protocolo de camada de aplicação que define o modelo de comunicação na *World Wide Web*. Operando sobre TCP na porta 80 (ou 8080, como neste trabalho), o HTTP segue o paradigma requisição-resposta: o cliente envia uma mensagem solicitando um recurso e o servidor responde com o conteúdo solicitado ou com um código de status indicando o resultado da operação.

No HTTP/1.1, as mensagens são compostas por uma linha inicial (*request line* ou *status line*), seguida por cabeçalhos no formato `Chave: Valor` e, opcionalmente, um corpo. Uma requisição GET típica possui a seguinte estrutura:

```
GET /arquivo.html HTTP/1.1
Host: www.exemplo.com
Connection: close
```

O servidor, por sua vez, responde com uma linha de status contendo o código numérico (200 para sucesso, 404 para recurso não encontrado, 403 para acesso proibido, entre outros), seguida pelos cabeçalhos e pelo corpo do recurso solicitado. O cabeçalho `Content-Length` é utilizado para informar ao cliente o tamanho exato do corpo, permitindo que este saiba quando a transferência foi concluída.

Neste trabalho, o miniservidor web implementa HTTP/1.1 de forma simplificada: suporta apenas o método GET, serve arquivos estáticos a partir de um diretório raiz (`www/`) e inclui um cabeçalho de autenticação `X-Custom-Auth` em todas as mensagens, um token SHA-256 contendo matrícula e nome do aluno, que permite identificar e validar os fluxos de interesse nas capturas de rede realizadas com TCPDump/Wireshark. Tanto o servidor quanto o cliente foram implementados para operar em dois modos de transporte distintos: sobre TCP nativo ou sobre R-UDP.

2.3. Protocolo TCP (Transmission Control Protocol)

O Protocolo de Controle de Transmissão (TCP) é um dos principais protocolos da camada de transporte da pilha TCP/IP, oferecendo um serviço orientado à conexão, confiável e com entrega ordenada de dados. Antes de qualquer transmissão, o TCP estabelece uma conexão lógica entre as partes por meio do mecanismo de *three-way handshake*, no qual são negociados números de sequência iniciais e parâmetros de janela.

Na transmissão, cada segmento enviado é numerado sequencialmente e confirmado por meio de ACKs cumulativos. Se um segmento não for confirmado dentro de um intervalo de tempo chamado RTO (*Retransmission Timeout*), calculado de forma adaptativa com base no RTT (*Round-Trip Time*) estimado, o TCP o retransmite automaticamente.

O controle de fluxo é realizado por meio de uma janela deslizante, na qual o receptor anuncia o tamanho de sua janela de recepção, limitando a quantidade de dados que

o transmissor pode enviar sem confirmação. Já o controle de congestionamento emprega algoritmos como *Slow Start*, *Congestion Avoidance* e *Fast Retransmit/Fast Recovery*, que ajustam dinamicamente a janela de congestionamento. Adicionalmente, o TCP utiliza um *checksum* de 16 bits para detecção de erros no cabeçalho e no *payload* de cada segmento.

2.3.1. R-UDP (UDP Confiável)

O R-UDP consiste em uma camada de confiabilidade implementada na própria aplicação sobre o protocolo UDP, adicionando mecanismos de entrega confiável e ordenada à comunicação originalmente não confiável do UDP. Neste trabalho, o R-UDP emprega o algoritmo *Stop-and-Wait*, o mais simples dos mecanismos de controle de fluxo, no qual o transmissor envia um pacote por vez e aguarda um ACK do receptor antes de enviar o próximo. Cada pacote possui um número de sequência de 32 bits, permitindo que o receptor detecte duplicatas. Caso um ACK se perca e o transmissor reenvie um pacote já recebido, o receptor o identifica pelo número de sequência, descarta o dado duplicado e reenvia o ACK correspondente.

3. Metodologia

Esta seção descreve a metodologia adotada para a coleta e análise dos dados experimentais. A seção 3.1 detalha o ambiente de execução, incluindo a infraestrutura Docker, a configuração de rede e os parâmetros de hardware e software utilizados. A seção 3.2 apresenta os cenários de rede simulados e o procedimento de testes. Por fim, a seção 3.3 define as métricas coletadas e descreve os métodos empregados para seu tratamento estatístico e validação cruzada com capturas de pacotes.

3.1. Ambiente de Execução

Os experimentos foram conduzidos em ambiente containerizado utilizando Docker Compose v2, com três *containers*, servidor DNS, servidor web e cliente web, interconectados por uma rede *bridge* interna no intervalo 172.28.0.0/16. Os *containers* compartilham um volume montado em `./results:/data` que persiste os resultados no hospedeiro. A Tabela 1 resume a configuração de cada *container*.

Tabela 1. Componentes do Ambiente

Componente	Função	Imagem	Porta	Privilégios
DNS	Resolução de nomes	Ubuntu 22.04	53/UDP	Nenhum
Servidor Web	Servir arquivos HTTP/1.1	Ubuntu 22.04	8080/TCP+UDP	NET_ADMIN
Cliente Web	DNS + HTTP GET	Ubuntu 22.04	Nenhuma	Nenhum

O servidor DNS, baseado em Ubuntu 22.04, opera na porta 53/UDP sem privilégios especiais, sendo responsável pela resolução de nomes da zona local, mapeando o domínio `www.web.local` para o IP 172.28.0.10. O servidor Web, também em Ubuntu 22.04, escuta na porta 8080 nos protocolos TCP e UDP, exposta para ambos os contêineres, e possui a *capability* NET_ADMIN, necessária para aplicar regras de simulação de rede via `tc qdisc` em sua interface, permitindo a inserção controlada de atraso e perda de pacotes. O cliente Web, por sua vez, executa a sequência completa de consulta

DNS (UDP) seguida de requisição HTTP GET (via TCP ou R-UDP) ao servidor Web, sem portas expostas e sem privilégios especiais.

A Tabela 2 detalha as ferramentas e bibliotecas utilizadas no ambiente experimental.

Tabela 2. Ferramentas e Bibliotecas		
Ferramenta	Versão	Função
Python	3.10	Linguagem de implementação
Docker Compose	v2	Orquestração dos containers
iproute2	–	Configuração de fila com tc qdisc netem
tcpdump	–	Captura de pacotes no servidor
tshark	–	Extração de métricas dos arquivos .pcap
pandas	≥ 2.0.0	Análise estatística dos resultados
matplotlib	≥ 3.7.0	Geração dos gráficos comparativos
python-dotenv	≥ 1.0.0	Carregamento das variáveis de ambiente

A Tabela 3 apresenta a configuração da máquina utilizada para a execução dos testes, fornecendo dados essenciais para a reprodutibilidade dos resultados. Essas informações permitem contextualizar os resultados obtidos, evidenciando as condições sob as quais os experimentos foram realizados.

Tabela 3. Especificações Técnicas da Máquina Utilizada nos Testes	
Especificação	Descrição
Processador	Intel Core i5-12450H 12º Gen (2.00 GHz) (8 núcleos, 12 threads, 12 MB cache)
Memória RAM	16,0 GB @ 3200 MHz (utilizável: 15,7 GB)
SO	Windows 11 Home Single Language (Executado no Ubuntu 24.04.3 LTS via WSL2)

3.2. Cenários de Teste

Para avaliar o impacto das condições de rede no desempenho da transferência de arquivos, foram definidos três cenários de simulação aplicados por meio da ferramenta `tc qdisc netem` na interface de rede do servidor Web. Cada cenário combina diferentes valores de atraso (*delay*) e perda de pacotes (*loss*), permitindo analisar o comportamento dos protocolos TCP e R-UDP sob condições progressivamente mais adversas.

O experimento cobre todas as combinações possíveis entre modo de transporte (TCP e R-UDP), cenário de rede (A, B e C) e tamanho de arquivo (100KB, 500KB e 1MB), totalizando 18 combinações distintas, conforme detalhado na Tabela 4. Para cada combinação, são executadas 10 repetições, resultando em 180 execuções no total.

Tabela 4. Combinações de parâmetros do experimento	
Protocolos	TCP e R-UDP
Cenário A	10 ms de atraso, 0% de perda
Cenário B	50 ms de atraso, 5% de perda
Cenário C	100 ms de atraso, 10% de perda
Arquivos	100 KB, 500 KB e 1 MB

3.3. Métricas Coletadas

Três métricas principais foram extraídas de cada rodada. Cada uma delas é registrada simultaneamente por duas fontes independentes: a aplicação cliente, ao final da transferência, e a captura de pacotes (TCPDump) executada no servidor, permitindo a validação cruzada dos resultados. Observe a Tabela 5.

Tabela 5. Métricas Utilizadas na Avaliação

Métrica	Dimensão	Descrição Básica	Fórmula
Tempo de Transferência	s	Intervalo de tempo decorrido entre o início e o fim da transmissão	$\Delta t = t_{\text{fim}} - t_{\text{início}}$
Taxa de Transferência (Vazão/Throughput)	Mbps	Volume de dados transmitidos por unidade de tempo	$T = \frac{D}{\Delta t}$
Taxa de Erro (Retransmissões)	%	Percentual de pacotes retransmitidos em relação ao total de pacotes enviados	$E = \frac{R}{P} \times 100$

Onde: Δt representa o tempo de transferência; $t_{\text{início}}$ e t_{fim} correspondem aos instantes de início e término da transmissão, respectivamente; D representa a quantidade de dados transmitidos (em bits); R representa o número de retransmissões observadas; e P representa o total de pacotes enviados.

4. Resultados

Nesta seção, são apresentados e analisados os dados obtidos nos testes de desempenho dos protocolos de transporte TCP e R-UDP aplicados à transferência de arquivos HTTP/1.1 em uma arquitetura cliente-servidor com resolução DNS prévia e com diferentes condições de qualidade de enlace. Além disso, os resultados são discutidos à luz dos fundamentos teóricos de cada abordagem, buscando relacionar o comportamento observado nos experimentos com as características esperadas em termos de tempo de transferência, vazão e taxa de erro.

4.1. Tempo Médio de Transferência

O Gráfico 1 apresenta a duração média total de cada execução, compreendendo a resolução DNS e a transferência HTTP, agregada por modo de transporte, cenário de rede e tamanho de arquivo. No cenário A, o TCP completou a transferência de 100KB em apenas 0,13s, enquanto o R-UDP levou 2,33s, uma diferença de 17 vezes. Para o arquivo de 1MB, a disparidade se ampliou: o TCP gastou 0,26s contra 22,73s do R-UDP, tornando o R-UDP 87 vezes mais lento. Esse comportamento é inerente ao *stop-and-wait*: a cada quadro transmitido, o protocolo aguarda a confirmação antes de prosseguir, acumulando o RTT do enlace a cada iteração. Com 1MB, o número de quadros é muito maior que com 100KB, e a penalidade cresce proporcionalmente.

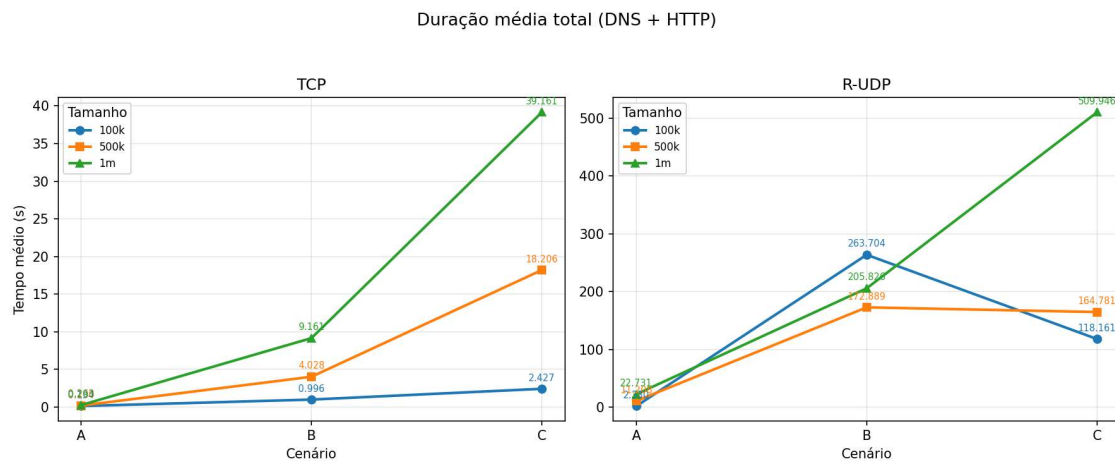


Figura 1. Comparação entre TCP e R-UDP: Tempo Médio de Transferência por Cenário

Nos cenários B e C, a diferença tornou-se ainda mais dramática. Para 100KB no cenário B, o TCP levou 1,0s contra impressionantes 263,7s do R-UDP, aproximadamente 265 vezes mais lento. Mesmo para 1MB, o R-UDP (205,8s) foi 22 vezes mais lento que o TCP (9,2s). No cenário C, o TCP para 1MB saltou para 39,2s (149 vezes mais lento que no cenário A), enquanto o R-UDP atingiu 509,9s, mais de 8 minutos para transferir 1MB, sendo 13 vezes mais lento que o TCP na mesma condição. Vale notar que, no pior cenário, o TCP conseguiu transferir 1MB (39,2s) em menos tempo que o R-UDP levou para transferir apenas 100KB no melhor cenário (2,33s), evidenciando a fragilidade do R-UDP mesmo em condições ideais de rede.

4.2. Vazão (Throughput)

As subsubseções seguintes são destinadas à análise das medidas estatísticas voltadas à vazão nos testes realizados.

4.2.1. Média

O Gráfico 2 apresenta a vazão média (*throughput*) em Mbps para cada combinação de modo, cenário e tamanho de arquivo. No cenário A, o TCP atingiu 6,25Mbps para 100KB e escalou para 31,92Mbps em 1MB, um ganho de 5 vezes entre o menor e o maior arquivo, reflexo do regime de janela deslizante que, quanto mais tempo permanece ativo, maior a vazão estabilizada. O R-UDP, ao contrário, manteve-se praticamente constante em 0,35–0,37Mbps independentemente do tamanho, evidenciando a limitação intrínseca do *stop-and-wait*: a vazão é governada pelo RTT e não se beneficia de transferências mais longas. Com isso, o TCP foi de 18 vezes mais rápido (100KB) a 86 vezes mais rápido (1MB) que o R-UDP no cenário A.

Nos cenários B e C, a vazão de ambos os protocolos despencou, mas o TCP manteve vantagem expressiva. No cenário B, o TCP entregou 1,10Mbps (100KB) a 1,29Mbps (1MB), enquanto o R-UDP caiu para 0,042–0,048Mbps, uma diferença de 22 a 40 vezes. No cenário C, o TCP variou de 0,24 a 0,46Mbps, e o R-UDP oscilou entre 0,019 e 0,025Mbps, com o TCP sendo de 10 a 22 vezes superior. Comparando o melhor com

o pior cenário, o TCP para 1MB sofreu uma redução de 97 vezes na vazão (de 31,92 para 0,33Mbps), enquanto o R-UDP para o mesmo arquivo caiu 19 vezes (de 0,37 para 0,019Mbps). Em termos absolutos, porém, a vazão do TCP no pior cenário (0,33Mbps) ainda superou a do R-UDP no melhor cenário (0,37Mbps) para o mesmo arquivo de 1MB.

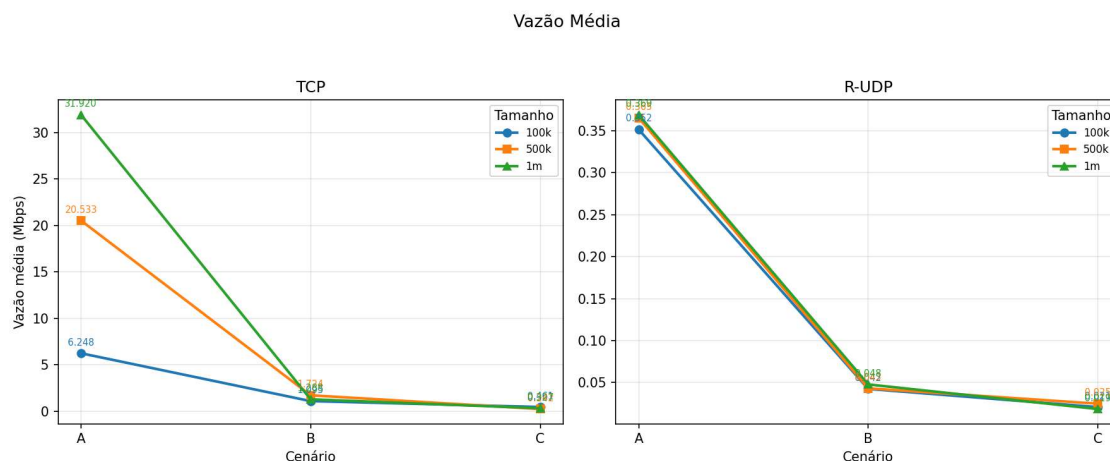


Figura 2. Comparação entre TCP e R-UDP: Vazão Média por Cenário

4.2.2. Vazão Mínima

A vazão mínima observada no Gráfico 3 em cada combinação revela o comportamento dos protocolos nos momentos mais adversos de cada bateria de 10 repetições. No cenário A, o TCP manteve mínimos elevados, 4,34 Mbps (100 KB), 19,48 Mbps (500 KB) e 30,93 Mbps (1 MB), muito próximos de suas respectivas médias, indicando baixa variabilidade em condições ideais. O R-UDP no mesmo cenário apresentou mínimos de 0,35 Mbps a 0,37 Mbps, igualmente estáveis e coerentes com sua média. A diferença entre os protocolos já era expressiva: o pior caso do TCP (4,34 Mbps) foi 12 vezes superior ao melhor caso do R-UDP (0,37 Mbps).

Nos cenários com perda, a diferença tornou-se abissal. No cenário B, o TCP registrou mínimos de 0,29 Mbps (100 KB) a 0,59 Mbps (1 MB), enquanto o R-UDP despencou para 0,0004 Mbps (100 KB), equivalentes a meros 411 bps, ou seja, 706 vezes menor que o mínimo do TCP para o mesmo arquivo. No cenário C, o TCP manteve mínimos entre 0,15 e 0,17 Mbps, ao passo que o R-UDP atingiu 0,00095 Mbps (100 KB), 158 vezes menor. Esses valores ínfimos no R-UDP refletem execuções em que múltiplos *timeouts* consecutivos de 500 ms ocorreram devido às perdas elevadas, praticamente paralisando a transferência. Em contraste, o TCP, mesmo em seu pior mínimo (0,15 Mbps no cenário C), ainda entregou vazão superior ao R-UDP no cenário A (0,37 Mbps) para o arquivo de 1 MB.

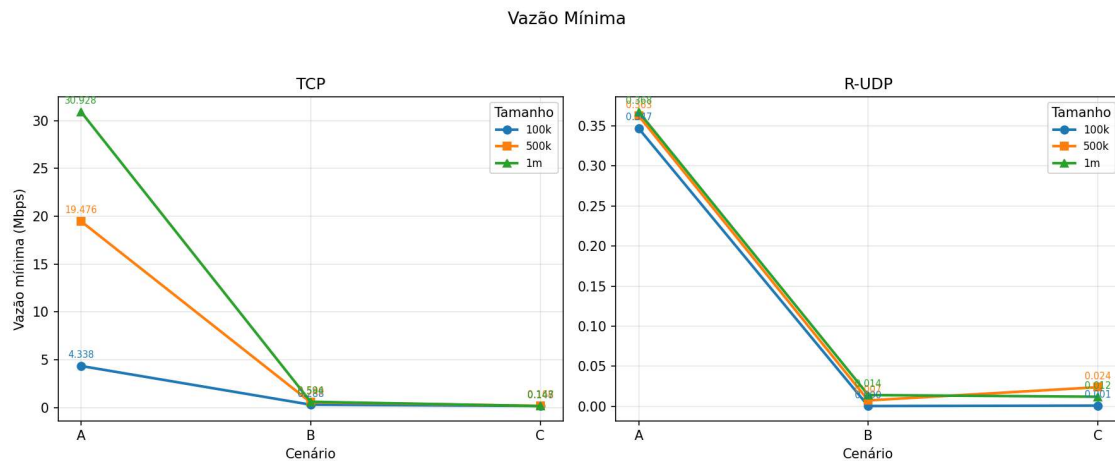


Figura 3. Comparação entre TCP e R-UDP: Vazão Mínima por Cenário

4.2.3. Vazão Máxima

A vazão máxima registrada no Gráfico 4 em cada bateria representa o melhor instante de cada combinação, livre de perdas ou *timeouts* excessivos. No cenário A, o TCP atingiu 7,19 Mbps (100 KB), 21,56 Mbps (500 KB) e 35,91 Mbps (1 MB), este último 97 vezes superior ao máximo do R-UDP no mesmo cenário (0,37 Mbps). O R-UDP, por sua vez, apresentou máximos praticamente iguais às suas médias em todos os tamanhos, confirmando que mesmo na melhor execução o *stop-and-wait* não consegue superar a limitação imposta pelo RTT do enlace.

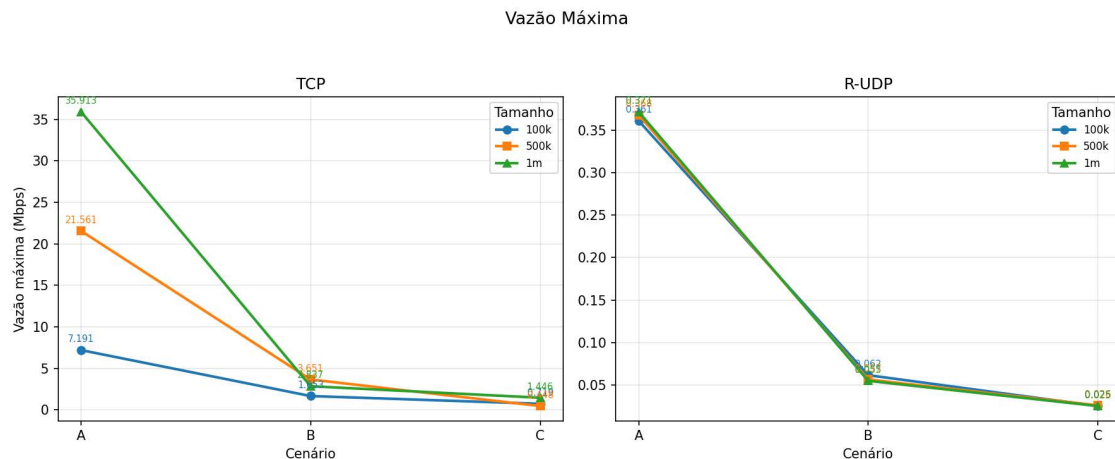


Figura 4. Comparação entre TCP e R-UDP: Vazão Máxima por Cenário

Nos cenários degradados, o TCP manteve máximos razoáveis: 1,65 Mbps (B, 100 KB) e 1,45 Mbps (C, 1 MB), enquanto o R-UDP atingiu apenas 0,062 Mbps (B, 100 KB) e 0,026 Mbps (C, 500 KB), diferenças de 26 e 56 vezes, respectivamente. Um contraste interessante: o pior máximo do TCP (0,45 Mbps no cenário C, 500 KB) ainda superou o melhor máximo do R-UDP em qualquer cenário com perda (0,062 Mbps no cenário B), embora tenha ficado abaixo do R-UDP no cenário A (0,37 Mbps). Observa-se também

que, à medida que a perda aumenta, o máximo do R-UDP se aproxima de sua média e mínimo, indicando que, diferentemente do TCP, onde uma execução favorecida pode escapar de perdas e atingir picos mais altos, no R-UDP todas as execuções são igualmente penalizadas pelo *timeout* fixo de 500 ms.

4.2.4. Desvio Padrão

No cenário A, ambos os protocolos apresentaram baixíssima dispersão: o TCP registrou CV (coeficiente de variação) de 4,7% para 1 MB e 13,1% para 100 KB, enquanto o R-UDP atingiu meros 0,3% a 1,2%, praticamente sem variação entre as 10 repetições. Isso confirma que, sem perda de pacotes, o comportamento de ambos é determinístico e previsível, o TCP estabiliza a janela e o R-UDP simplesmente repete o mesmo ciclo *stop-and-wait* a cada execução.

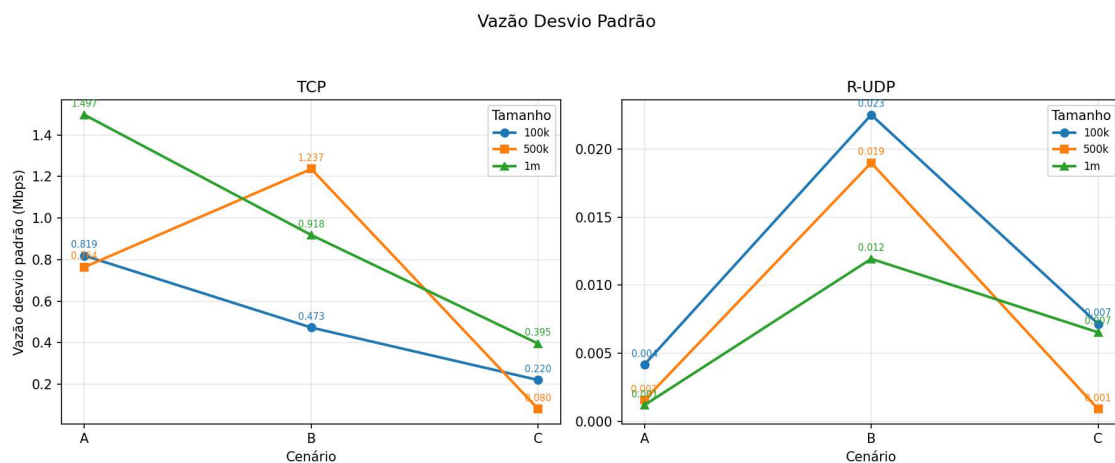


Figura 5. Comparação entre TCP e R-UDP: Desvio Padrão da Vazão por Cenário

Nos cenários com perda, a variabilidade disparou. No cenário B, o TCP apresentou CV entre 43% e 72%, e o R-UDP entre 25% e 55%, reflexo direto da aleatoriedade das perdas: uma execução que sofre poucas perdas converge rápido, enquanto outra com múltiplas perdas consecutivas pode demorar muito mais. O caso mais extremo ocorreu no cenário C, onde o TCP para 1 MB registrou desvio padrão de 0,40 Mbps contra uma média de apenas 0,33 Mbps, um CV de 121%, indicando que o desvio supera a própria média. Isso ocorre porque, em algumas repetições, o TCP consegue escapar das perdas e manter vazão razoável, enquanto em outras o controle de congestionamento reduz drasticamente a janela, gerando uma dispersão muito alta. O R-UDP no mesmo cenário para 500 KB, curiosamente, teve o menor CV entre todos os cenários com perda (3,6%), sugerindo que, para esse tamanho de arquivo, o número de *timeouts* foi consistente entre as repetições.

4.3. Taxa de Erro (Retransmissões)

No Cenário A, tanto o TCP quanto o R-UDP apresentaram taxa de erro de 0,00% em todas as execuções, conforme esperado, já que não há perda de pacotes simulada. No Cenário B, o TCP apresentou uma taxa média de apenas 0,13%, com valor máximo de

1,37%. Já o R-UDP apresentou uma média de 20,72%. Excluindo casos extremos apresentados nas execuções, a taxa do R-UDP fica em torno de 9,5%, o que é condizente com o comportamento do *Stop-and-Wait*, onde cada pacote perdido gera uma retransmissão e essa retransmissão também pode ser perdida, criando um efeito acumulativo.

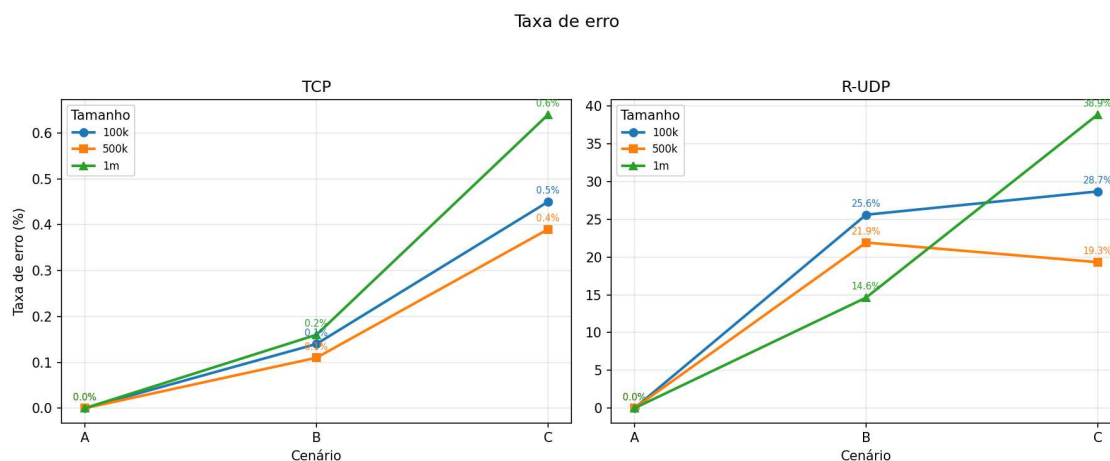


Figura 6. Comparação entre TCP e R-UDP: Taxa de Erro de Transmissão por Cenário

No Cenário C, o TCP manteve uma taxa média baixa de 0,49%, com máximo de 3,08%. O R-UDP apresentou média de 28,95%, novamente puxada para cima por outliers com taxas altas. Sem esses outliers, a taxa se situa entre 16% e 24%. A diferença expressiva entre TCP e R-UDP se explica por um fator: o TCP é inerentemente mais eficiente: utiliza ACKs cumulativos, janela deslizante e SACK, conseguindo recuperar múltiplas perdas com poucas retransmissões.

5. Discussão

Esta seção destina-se a responder os seguintes questionamentos:

1. Como a perda de pacotes simulada no canal afetou o tempo de resolução DNS (que usa UDP nativo sem retransmissão na camada de transporte) em comparação com o download da página via HTTP (R-UDP/TCP)? O cliente DNS precisou implementar algum *timeout* na aplicação?

Sim, o cliente DNS precisou implementar *timeout* e retransmissão na camada de aplicação (Figura 7 do Anexo I contido na página 15). Como o DNS utiliza UDP nativo (sem conexão e sem confiabilidade garantida pelo protocolo de transporte), é implementado um mecanismo próprio de tolerância a falhas: ele define um *timeout* de 2 segundos por tentativa e realiza até 5 retransmissões em caso de perda de pacote. Esse comportamento é essencial porque o UDP não oferece confirmação de entrega, se o pacote de consulta ou a resposta for perdido, a aplicação jamais saberia sem um mecanismo explícito de *timeout*. Quanto ao impacto da perda simulada sobre o tempo de resolução DNS em comparação com o download HTTP, é importante considerar que, neste experimento, as regras de `tc qdisc netem` foram aplicadas exclusivamente à interface de rede do servidor Web (no sentido de saída). Isso significa que o tráfego DNS entre o cliente e o servidor DNS trafega diretamente pela rede *bridge* do Docker sem atravessar a interface do servidor

Web, portanto não foi diretamente submetido ao atraso e à perda simulados. Consequentemente, o tempo de resolução DNS manteve-se baixo e estável (da ordem de milissegundos) em todos os cenários, contribuindo com fração desprezível da duração total, tipicamente menos de 1%, conforme discutido na análise da duração total. Já o download HTTP (seja via TCP ou R-UDP) sofreu integralmente os efeitos do atraso e da perda impostos, resultando em degradação severa do tempo de transferência e da vazão, especialmente nos cenários B e C para arquivos maiores.

2. Qual foi o impacto visual (na análise de gráficos) e métrico do *overhead* dos cabeçalhos HTTP adicionados nesta avaliação, quando comparado ao protocolo de aplicação puramente textual e customizado da Segunda Avaliação?

O *overhead* dos cabeçalhos HTTP é desprezível em termos de bytes, no máximo 0,33% para arquivos de 100KB e apenas 0,033% para 1MB. Em uma transferência de 1MB, os 341 bytes extras equivalem a menos que um único quadro R-UDP adicional (cujo *overhead* de *framing* já é de 93 bytes por quadro, independentemente do protocolo de aplicação). O custo real do HTTP não está nos bytes, mas na complexidade de *parsing* (cabeçalhos texto vs. binário) e na obrigatoriedade do X-Custom-Auth em cada mensagem, que já existia no enquadramento anterior e foi mantida para compatibilidade de análise no Wireshark.

3. Ao inspecionar o Wireshark, o fluxo de pacotes seguiu estritamente a ordem esperada da arquitetura da Internet? Demonstre através de *prints* do fluxo de tempo do Wireshark a transição exata entre o encerramento da *query* DNS e o início do *handshake* TCP / primeira transmissão R-UDP.

Sim. A análise das capturas no Wireshark mostrou que o fluxo de pacotes seguiu a ordem esperada da arquitetura da Internet. Em ambos os experimentos, a comunicação iniciou com a resolução de nomes via DNS, responsável por obter o endereço IP do destino. Após o encerramento das consultas DNS, observou-se o início da comunicação de transporte. No experimento utilizando TCP, a transição ocorreu para o *three-way handshake* (pacotes SYN, SYN/ACK e ACK), seguido pela primeira transmissão de dados da aplicação. Já no experimento utilizando R-UDP, por se tratar de uma solução baseada em UDP, não houve etapa de estabelecimento de conexão; assim, logo após a resolução DNS foi enviada a primeira mensagem da aplicação, iniciando a transferência de dados. Dessa forma, a Figura 8 apresentada no **Anexo II** (página 16), confirma que não houve transmissão de dados da aplicação antes da conclusão da resolução DNS, e que cada protocolo seguiu o comportamento esperado de sua respectiva camada de transporte.

6. Conclusão

Este trabalho implementou e comparou dois protocolos de transferência de arquivos, TCP e R-UDP (UDP com camada de confiabilidade), sob três cenários de rede simulados com tc netem: A (10ms, 0% de perda), B (50ms, 5%) e C (100ms, 10%). Foram realizadas 10 rodadas de transferência para cada combinação de protocolo, cenário de rede e tamanho de arquivo (100 KB, 500 KB e 1 MB), totalizando 180 execuções. A arquitetura integra cliente/servidor HTTP com suporte a resolução DNS local, permitindo análise comparativa do desempenho desses protocolos de transporte em redes com características progressivamente mais adversas.

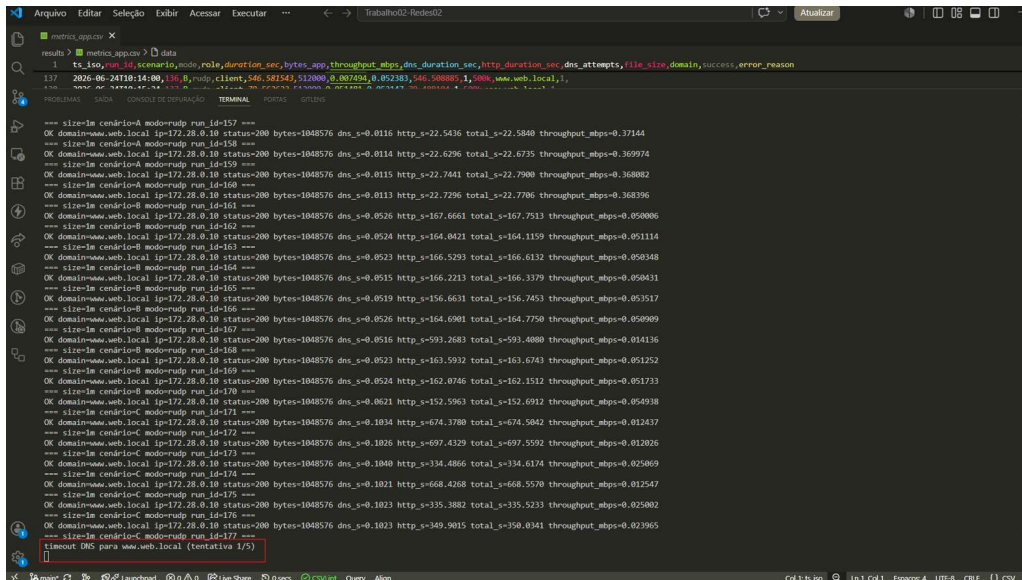
Embora R-UDP tenha demonstrado confiabilidade 100%, sua implementação

atual não é viável para transferência de dados em tempo real. O modelo de reconhecimento por quadro introduz latência cumulativa que aumenta dramaticamente com a latência de propagação da rede.

Este trabalho reforça um princípio fundamental das redes de computadores: o protocolo ideal depende da caracterização específica do cenário de uso. Embora R-UDP seja uma abordagem válida para fins educacionais e teóricos, os resultados confirmam que TCP continua sendo a solução mais eficiente para transferência de dados confiável em redes reais, especialmente quando consideramos o impacto combinado de latência e perda de pacotes.

Anexo I

Capturas Complementares: Ocorrência de um Retry com Timeout Durante Execuções de Transferências.



```
results > metrics_app.csv > data
117 2025-05-21T10:14:00.136.117 mod,rule,duration_sec,bytes_app,throughput_mbps,dns_duration_sec,http_duration_sec,dns_attempts,file_size,domain,success,error_reason
---
size:1m cenario:A modorudp run id=157 ---
OK domain:www.web.local ip=172.28.0.10 status=200 bytes=1048576 dns_s=0.0116 http_s=22.5436 total_s=22.5840 throughput_mbps=0.37144
size:1m cenario:A modorudp run id=158 ---
OK domain:www.web.local ip=172.28.0.10 status=200 bytes=1048576 dns_s=0.0114 http_s=22.6296 total_s=22.6735 throughput_mbps=0.369974
size:1m cenario:A modorudp run id=159 ---
OK domain:www.web.local ip=172.28.0.10 status=200 bytes=1048576 dns_s=0.0115 http_s=22.7441 total_s=22.7900 throughput_mbps=0.368802
size:1m cenario:A modorudp run id=160 ---
OK domain:www.web.local ip=172.28.0.10 status=200 bytes=1048576 dns_s=0.0113 http_s=22.7296 total_s=22.7706 throughput_mbps=0.368396
size:1m cenario:B modorudp run id=161 ---
OK domain:www.web.local ip=172.28.0.10 status=200 bytes=1048576 dns_s=0.0526 http_s=167.6661 total_s=167.7513 throughput_mbps=0.050006
size:1m cenario:B modorudp run id=162 ---
OK domain:www.web.local ip=172.28.0.10 status=200 bytes=1048576 dns_s=0.0524 http_s=164.9421 total_s=164.1159 throughput_mbps=0.051114
size:1m cenario:B modorudp run id=163 ---
OK domain:www.web.local ip=172.28.0.10 status=200 bytes=1048576 dns_s=0.0523 http_s=166.5293 total_s=166.6132 throughput_mbps=0.050348
size:1m cenario:B modorudp run id=164 ---
OK domain:www.web.local ip=172.28.0.10 status=200 bytes=1048576 dns_s=0.0515 http_s=166.2213 total_s=166.3379 throughput_mbps=0.050431
size:1m cenario:B modorudp run id=165 ---
OK domain:www.web.local ip=172.28.0.10 status=200 bytes=1048576 dns_s=0.0519 http_s=156.6631 total_s=156.7453 throughput_mbps=0.053517
size:1m cenario:B modorudp run id=166 ---
OK domain:www.web.local ip=172.28.0.10 status=200 bytes=1048576 dns_s=0.0526 http_s=164.6901 total_s=164.7750 throughput_mbps=0.050909
size:1m cenario:B modorudp run id=167 ---
OK domain:www.web.local ip=172.28.0.10 status=200 bytes=1048576 dns_s=0.0516 http_s=593.2683 total_s=593.4080 throughput_mbps=0.014136
size:1m cenario:B modorudp run id=168 ---
OK domain:www.web.local ip=172.28.0.10 status=200 bytes=1048576 dns_s=0.0523 http_s=163.5932 total_s=163.6743 throughput_mbps=0.051252
size:1m cenario:B modorudp run id=169 ---
OK domain:www.web.local ip=172.28.0.10 status=200 bytes=1048576 dns_s=0.0524 http_s=162.8746 total_s=162.1512 throughput_mbps=0.051733
size:1m cenario:B modorudp run id=170 ---
OK domain:www.web.local ip=172.28.0.10 status=200 bytes=1048576 dns_s=0.0621 http_s=152.5963 total_s=152.6912 throughput_mbps=0.054938
size:1m cenario:C modorudp run id=171 ---
OK domain:www.web.local ip=172.28.0.10 status=200 bytes=1048576 dns_s=0.1034 http_s=674.3780 total_s=674.5042 throughput_mbps=0.012437
size:1m cenario:C modorudp run id=172 ---
OK domain:www.web.local ip=172.28.0.10 status=200 bytes=1048576 dns_s=0.1026 http_s=697.4329 total_s=697.5592 throughput_mbps=0.012026
size:1m cenario:C modorudp run id=173 ---
OK domain:www.web.local ip=172.28.0.10 status=200 bytes=1048576 dns_s=0.1040 http_s=334.4866 total_s=334.6174 throughput_mbps=0.025069
size:1m cenario:C modorudp run id=174 ---
OK domain:www.web.local ip=172.28.0.10 status=200 bytes=1048576 dns_s=0.1021 http_s=668.4268 total_s=668.5570 throughput_mbps=0.012547
size:1m cenario:C modorudp run id=175 ---
OK domain:www.web.local ip=172.28.0.10 status=200 bytes=1048576 dns_s=0.1023 http_s=335.3882 total_s=335.5233 throughput_mbps=0.025002
size:1m cenario:C modorudp run id=176 ---
OK domain:www.web.local ip=172.28.0.10 status=200 bytes=1048576 dns_s=0.1023 http_s=349.5015 total_s=350.0341 throughput_mbps=0.021965
size:1m cenario:C modorudp run id=177 ---
timeout DNS para www.web.local (tentativa 1/5)
```

Figura 7. Atuação do Mecanismo de Retry com Timeout Durante Transferência de Arquivo

Anexo II

Capturas Complementares: Fluxo entre Pacotes no Wireshark.

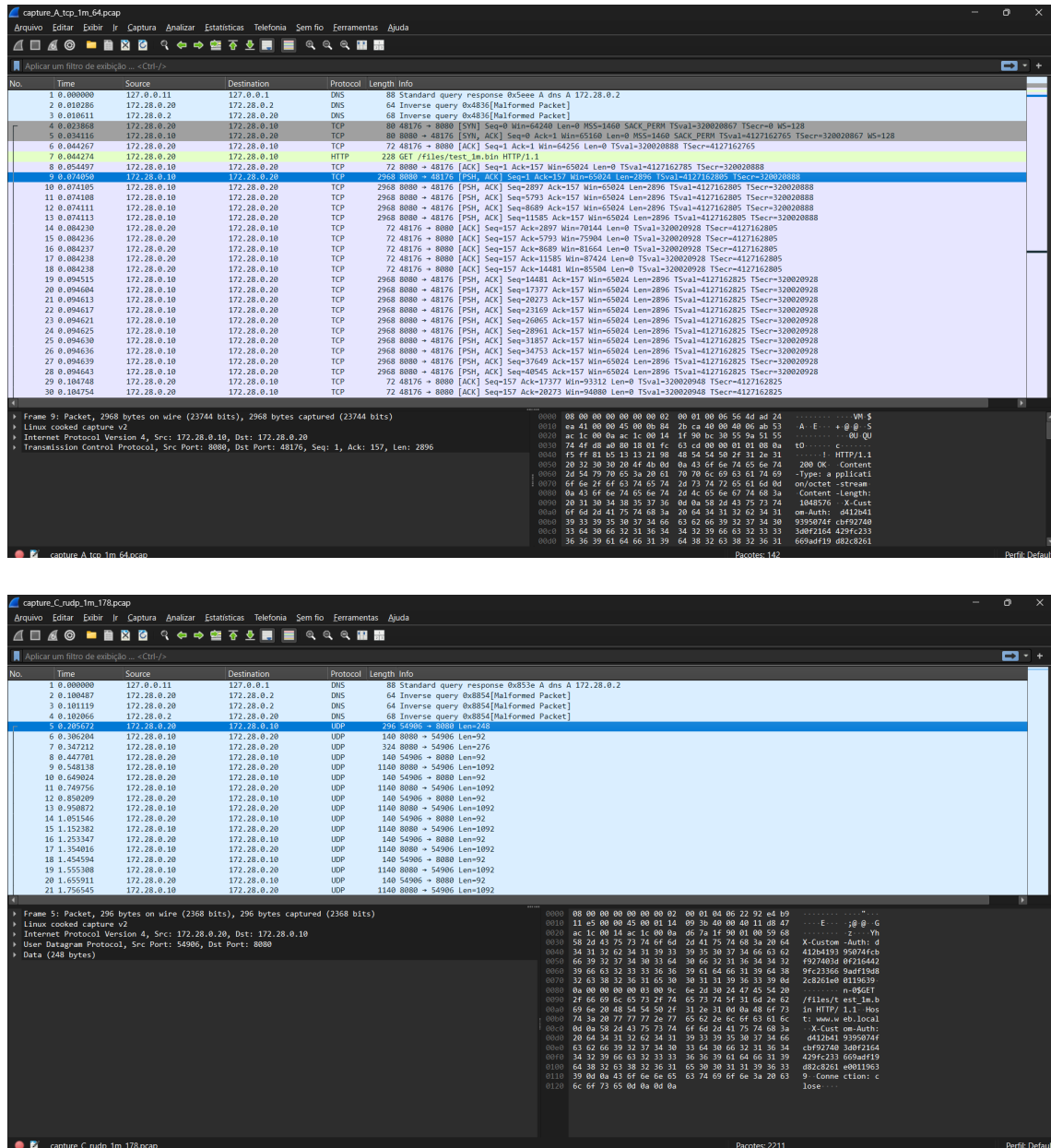


Figura 8. Fluxo de Pacotes Durante Transmissões TCP e R-UDP.